

§7: Sortieren in linearer Zeit

Untere Schranke · Counting Sort · Radix Sort · Bucket Sort

1 Untere Schranke für vergleichsbasiertes Sortieren

Satz 7.1 — Untere Schranke $\Omega(n \log n)$

Modell: Gegeben $a = (e_1, \dots, e_n)$; die einzige erlaubte Operation ist der Vergleich $e_i \leq e_j$?

Satz: Jeder vergleichsbasierte Sortieralgorithmus benötigt im *schlechtesten Fall* $\Omega(n \log n)$ Vergleiche.

Beweisidee (Entscheidungsbaum): Jeder Algorithmus entspricht einem binären Entscheidungsbaum mit einem Blatt pro möglicher Permutation. Es gibt $n!$ Permutationen, also mindestens $n!$ Blätter. Die Tiefe eines Baums mit $\geq n!$ Blättern beträgt mindestens $\lceil \log_2(n!) \rceil \geq \lceil n \log_2 n - n \log_2 e \rceil \in \Omega(n \log n)$ (Stirling: $\log_2(n!) \approx n \log_2 n - n \log_2 e$).

Konsequenz: Mergesort und Heapsort mit $\Theta(n \log n)$ sind *asymptotisch optimal* unter vergleichsbasierten Algorithmen.

Schranke brechen: Durch Nutzung zusätzlicher Struktureigenschaften der Schlüssel (z. B. beschränkter Wertebereich, Gleichverteilung) ist $O(n)$ erreichbar.

2 Counting Sort

Definition 7.2 — Counting Sort

Voraussetzung: $a[j] \in \mathbb{N}$, $0 \leq a[j] \leq M - 1$, $M \in O(n)$ (d. h. $M < n$).

Stabil. Idee: Zähle, wie oft jeder Wert vorkommt, berechne Präfixsummen als Startadressen, kopiere rückwärts in Ausgabe-Array.

Algorithmus (3 Phasen):

```
def countingSort(a, M):
    b = [0] * M                                # Zähler-Array, initialisiert mit 0

    # Phase 1: Häufigkeiten zählen
    for i in range(len(a)):
        b[a[i]] += 1

    # Phase 2: Präfixsummen = Startadressen
    for i in range(1, M):
        b[i] += b[i-1]

    # Phase 3: Rückwärts kopieren (sichert Stabilität)
    r = [0] * len(a)
    for i in range(len(a)-1, -1, -1):
        b[a[i]] -= 1
        r[b[a[i]]] = a[i]
    return r
```

Komplexität: $\Theta(n + M)$. Da $M \in O(n)$: $\Theta(n)$.

Speicher: $O(n + M)$ (zusätzliches Array der Länge n + Zähler-Array der Länge M).

Beispiel 7.3 — Counting Sort Schritt für Schritt ($M = 10$)

$a = [2, 5, 7, 5, 5, 2, 6, 1, 3, 1, 6, 6]$, $n = 12$, $M = 10$

Phase 1 — Häufigkeiten ($b[k] = \text{Anzahl Vorkommen von } k$):

Index k	0	1	2	3	4	5	6	7	8	9
$b[k]$	0	2	2	1	0	3	3	1	0	0

Phase 2 — Präfixsummen ($\text{int}[k] = \text{Anzahl Elemente } \leq k = \text{Endindex von } k$):

Index k	0	1	2	3	4	5	6	7	8	9
$\text{int}[k]$	0	2	4	5	5	8	11	12	12	12

Ergebnis: $r = [1, 1, 2, 2, 3, 5, 5, 5, 6, 6, 6, 7]$ ✓ (sortiert, stabil)

Definition 7.4 — Präfixsummen-Anwendung

Die Präfixsummen in Phase 2 erlauben auch Bereichsabfragen: $\sum_{i=\ell}^r b[i] = \text{int}[r] - \text{int}[\ell - 1]$

Verallgemeinerung auf 2D: Für ein 2D-Array summiert man zeilenweise ($\rightarrow$), dann spaltenweise ($\downarrow$).
Bereichssumme per Inklusion/Exklusion: $S(\ell_x:r_x, \ell_y:r_y) = \text{int}[r_x][r_y] - \text{int}[\ell_x-1][r_y] - \text{int}[r_x][\ell_y-1] + \text{int}[\ell_x-1][\ell_y-1]$

Achtung

Counting Sort ist nur **stabil**, weil Phase 3 rückwärts iteriert (`range(len(a)-1, -1, -1)`). Vorwärts-Iteration würde die Reihenfolge gleicher Elemente umkehren.

3 Radix Sort

Definition 7.5 — Radix Sort

Voraussetzung: $a[i] \in [0, M]$, $\text{len}(a) \leq M$.

Idee: Sortiere Ziffer für Ziffer, beginnend mit der *am wenigsten signifikanten* Stelle (Least-Significant-Digit first), mit einem *stabilen* Sortieralgorithmus.

k -te Ziffer zur Basis B :

$$x = d_n B^n + d_{n-1} B^{n-1} + \dots + d_0 \implies d_k = \left\lfloor \frac{x}{B^k} \right\rfloor \bmod B = (x // B^{**k}) \% B$$

Komplexität: d Durchläufe je $\Theta(m + n)$ (Counting Sort, $m = B$):

$$T = \Theta((m + n) \cdot d) = \Theta((m + n) \cdot \log_m M)$$

Optimale Wahl $m = 2$: $T = O(n \log_2 M)$ (implementiert mit Bit-Shifts $>>/<<$).

Praxisoptimum ($M = 2^{32}$): $m = 2^8 = 256$, $d = 4$ Durchläufe à $\Theta(256 + n)$.

Beispiel 7.6 — Radix Sort ($B = 10$, dreistellige Zahlen) $a = [329, 457, 657, 839, 436, 720, 355]$

Eingabe	nach Einerstelle	nach Zehnerstelle	nach Hunderterstelle
329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Beachte: Stabilität in jedem Schritt ist notwendig, damit frühere Sortierungen erhalten bleiben!

4 Bucket Sort

Definition 7.7 — Bucket Sort**Voraussetzung:** $a[i] \sim$ gleichmäßig auf $[0, 1]$ verteilt.**Algorithmus:**

1. Wähle $M \approx n/d$ Buckets, $d \in 1 \dots 10$.
2. Teile $[0, 1]$ auf M Buckets auf; Abbildung: $\text{bucketMap}(x, M) = \lfloor x \cdot M \rfloor$.
3. Verteile alle Elemente in ihre Buckets.
4. Sortiere jeden Bucket intern (z. B. mit Insertion Sort).
5. Konkateniere die sortierten Buckets.

```
def bucketMap(key, M):
    return int(key * M)

def bucketSort(a, M):
    buckets = [[] for k in range(M)]
    for k in range(len(a)):
        buckets[bucketMap(a[k], M)].append(a[k])
    start = 0
    for k in range(M):
        insertionSort(buckets[k])
        end = start + len(buckets[k])
        a[start:end] = buckets[k]
        start += len(buckets[k])
```

Komplexität (durchschnittlich):

- Verteilen: $\Theta(n)$
- Internes Sortieren: $O(M \cdot d^2)$, wobei $d = n/M$ die erwartete Bucket-Größe ist
- Konkatenieren: $\Theta(n)$
- Gesamt: $O(n + Md^2) \Rightarrow O(n)$, wenn $Md^2 \in O(n)$

Stabil (wenn internes Sortieren stabil ist). **Nicht in-place.**

5 Vergleich der Algorithmen

	Counting Sort	Radix Sort	Bucket Sort	Mergesort (Ref.)
Zeitkomplexität	$\Theta(n + M)$	$\Theta((m + n) \log_m M)$	$O(n)$ avg	$\Theta(n \log n)$
Speicher	$O(n + M)$	$O(n + m)$	$O(n + M)$	$O(n)$
Stabil?	Ja	Ja (wenn CS)	Ja	Ja
Eingabe	$a[i] \in [0, M)$ ganzzahlig	$a[i] \in [0, M]$	$a[i] \in [0, 1]$ gleichvt.	beliebig vergleichbar
In-place?	Nein	Nein	Nein	Nein
Wann optimal?	$M \in O(n)$	$m = 2$: $O(n \log_2 M)$	Gleichverteilung	Allgemeinfall

Verfahren	Kernidee	Warum schneller als $\Omega(n \log n)$?
Counting Sort	Zählen + Präfixsummen	Kein Vergleich; Wertebereich $[0, M)$ bekannt
Radix Sort	Stellenweise stabil sortieren	Schlüssel als Ziffernfolge, nicht als atomarer Wert
Bucket Sort	Gleichvert. $\Rightarrow$ homogene Buckets	Verteilungsannahme ersetzt Vergleichsinformation

Prüfungsscheckliste — §7 Sortieren in linearer Zeit

- ☐ Untere Schranke $\Omega(n \log n)$ für vergleichsbasiertes Sortieren erklären: $\rightarrow$ *Satz 7.1*
Entscheidungsbaum-Argument, $n!$ Blätter, Tiefe $\geq \log_2(n!)$.
- ☐ Warum können Counting/Radix/Bucket Sort die Schranke brechen? $\rightarrow$ *Satz 7.1*
Welche zusätzliche Annahme wird jeweils gemacht?
- ☐ Counting Sort: Voraussetzungen nennen ($a[j] \in [0, M)$, $M \in O(n)$). Alle $\rightarrow$ *Def. 7.2*
drei Phasen auswendig können.
- ☐ Warum muss Phase 3 rückwärts laufen? Was würde passieren, wenn vor- $\rightarrow$ *Warn.*
wärts iteriert wird?
- ☐ Counting Sort vollständig für ein konkretes Beispiel durchführen: b - $\rightarrow$ *Bsp. 7.3*
Array, int-Array, Ergebnis.
- ☐ Komplexität von Counting Sort herleiten: $\Theta(n + M)$; wann $\Theta(n)$? $\rightarrow$ *Def. 7.2*
- ☐ Präfixsummen-Trick erklären: Bereichsabfrage $\sum_{i=\ell}^r b[i]$ in $O(1)$ nach $\rightarrow$ *Def. 7.4*
 $O(M)$ Vorverarbeitung.
- ☐ Radix Sort: Idee (LSB-first), Formel für die k -te Ziffer $d_k = \lfloor x/B^k \rfloor \bmod B$, Python: `(x // B**k) % B`. $\rightarrow$ *Def. 7.5*
- ☐ Warum muss der in Radix Sort verwendete Sortieralgorithmus stabil $\rightarrow$ *Def. 7.5*
sein? Gegenbeispiel skizzieren.
- ☐ Komplexität Radix Sort: $\Theta((m + n) \log_m M)$; optimales $m = 2$ ergibt $\rightarrow$ *Def. 7.5*
 $O(n \log_2 M)$.
- ☐ Radix Sort für 3-stellige Zahlen (Basis 10) vollständig durchführen: drei $\rightarrow$ *Bsp. 7.6*
Sortierdurchläufe zeigen.
- ☐ Bucket Sort: Voraussetzung (Gleichverteilung), Algorithmus (4 Schritte), $\rightarrow$ *Def. 7.7*
Python-Code kennen.
- ☐ Durchschnittliche Komplexität Bucket Sort herleiten: $O(n + Md^2) =$ $\rightarrow$ *Def. 7.7*
 $O(n)$ wenn $Md^2 \in O(n)$.